

HARPIA

CONTEXTO COMPLETO PARA IA

People & Internal Operations OS

Produto, arquitetura, dominios, IA, workflows e
diretrizes para continuar o código com segurança.

Harpia - contexto completo para IA

Documento criado para explicar o produto, a proposta, a arquitetura e os pontos principais do código para outra IA conseguir entender rapidamente o projeto e continuar trabalhando nele com segurança.

1. Resumo executivo

Harpia é um SaaS B2B para People Ops e Operações Internas. A proposta central é transformar RH, atendimento interno, onboarding, tarefas, conhecimento, aprovações e recrutamento em um sistema operacional único, rápido e assistido por IA.

O produto nasceu de uma base de recrutamento chamada HireFlow AI, mas evoluiu para algo maior: um People & Internal Operations OS. Hoje o recrutamento ainda existe e continua importante, mas virou módulo complementar. O centro do produto passou a ser operação diária: colaboradores, solicitações internas, tarefas, workflows, compliance, conhecimento, chat corporativo, automações e IA com aprovação segura.

Pitch curto:

Harpia é um sistema operacional interno para times de People Ops que reduz ruído, organiza demandas, recomenda próximas ações e permite que a IA execute trabalho real com rastreabilidade e aprovação.

2. Proposta de valor

O Harpia resolve um problema comum em empresas em crescimento: informações de pessoas, solicitações, tarefas e decisões ficam espalhadas em planilhas, Slack, Notion, email, ferramentas de recrutamento e sistemas de RH pouco integrados.

O produto busca entregar:

- Uma central única para operação de pessoas.
- Filas claras de solicitações, tarefas e aprovações.
- Workflows estruturados de onboarding e offboarding.
- Base de conhecimento conectada ao contexto operacional.
- Chat corporativo que entende dados internos e pode propor ações.
- IA integrada ao fluxo, não apenas um chat separado.
- Auditoria e aprovação para ações sensíveis.
- Recruiting preservado como módulo de hiring, com candidatos, vagas, pipeline e entrevistas.
- Produto com experiência premium, limpa, rápida e intuitiva.

3. Público-alvo

Usuários principais:

- People Ops.
- RH operacional.
- Managers.

- Founders.
- Times de recrutamento.
- Operacoes internas.
- Financeiro ou administracao quando envolve billing, aprovacoes ou requisicoes internas.

Comprador provavel:

- Founder ou COO de startup em crescimento.
- Head de People.
- Diretor de Operacoes.
- Lider de RH em empresa que precisa profissionalizar processos internos.

O produto deve parecer confiavel para compra B2B: organizado, auditavel, rapido, seguro e facil para usuarios nao tecnicos.

4. Mentalidade de produto

Harpia nao deve parecer uma colecao de telas CRUD. Ele deve parecer um sistema operacional de decisao.

Principios importantes:

- Menos navegacao, mais acao.
- IA como acelerador de workflow, nao como enfeite.
- Tudo que a IA sugerir deve explicar motivo, risco, registros afetados e possibilidade de aprovacao.
- Pessoas leigas precisam entender o que fazer sem treinamento longo.
- Listas e detalhes devem ser obvios: o usuario escolhe um item, entende contexto, toma uma acao.
- O chat deve operar o produto, criando tarefas, respondendo solicitacoes, movendo etapas, gerando resumos e preparando acoes.
- O produto deve funcionar bem em tema claro e escuro.
- Evitar excesso de frames, bordas arredondadas, cards pesados e canvas decorativos.
- Preferir interface simples, plana, limpa e funcional, inspirada em Vercel, ChatGPT, Notion, Linear e Trello.

5. Stack tecnica

Stack principal:

- Next.js App Router.
- React 19.
- TypeScript.
- Tailwind CSS.
- Prisma.
- PostgreSQL.
- NextAuth/Auth.js com provider de credenciais.
- Zod para validacao.
- React Hook Form em formularios.
- Lucide React para icones.
- Framer Motion em algumas experiencias visuais.

- pdf-lib para geracao de PDFs.
- OpenAI SDK com suporte tambem ao endpoint OpenAI-compatible do Gemini.
- Tauri + Vite + React para cliente desktop em `apps/desktop`.

Scripts importantes:

- `npm run dev`: sobe o Next em desenvolvimento.
- `npm run build`: gera Prisma Client e builda Next.
- `npm run lint`: roda ESLint.
- `npm run typecheck`: roda TypeScript sem emitir arquivos.
- `npm run verify`: lint + typecheck.
- `npm run db:generate`: gera Prisma Client.
- `npm run db:push`: aplica schema no banco via Prisma.
- `npm run db:seed`: popula dados demo.
- `npm run jobs:process`: processa background jobs.

Observacao operacional:

Em Windows local, o build pode compilar e depois cair com `spawn EPERM`. A Vercel ja validou build cloud corretamente em deploys anteriores. Para mudancas importantes, sempre rode pelo menos `typecheck` e `lint`.

6. Estrutura do repositorio

Pastas principais:

- `app/`: rotas Next.js App Router.
- `app/(marketing)/`: landing page, pricing, book demo e paginas publicas de carreira.
- `app/(auth)/`: login e invite.
- `app/(app)/`: area autenticada do produto.
- `app/api/`: endpoints de API, auth, health, cron, desktop, PDF demo e webhooks.
- `components/`: componentes reutilizaveis, UI, layout, chat, people, operations, dashboard, AI e automations.
- `lib/`: utilitarios compartilhados, auth, billing, AI, queries, storage, observabilidade e validacoes.
- `modules/`: dominios do produto com queries, services, validators e runtimes.
- `prisma/`: schema e seed.
- `apps/desktop/`: cliente desktop.
- `docs/`: documentacao de arquitetura, auditorias e contexto.
- `scripts/`: scripts operacionais, incluindo jobs.
- `public/`: assets publicos.

Padrao mental do codigo:

- `app/` monta paginas e chama permissoes, queries e componentes.
- `modules/\*/queries.ts` contem leituras por dominio.
- `modules/\*/service.ts` contem comandos e mutacoes por dominio.
- `modules/\*/validators.ts` contem schemas de entrada quando aplicavel.
- `lib/\*` contem infraestrutura, helpers e queries historicas compartilhadas.
- `components/\*` contem UI, normalmente separada por dominio ou experiencia.

7. Rotas principais do produto

Area publica:

- ``/``: marketing home.
- ``/pricing``: precos.
- ``/book-demo``: demonstracao.
- ``/careers/[slug]``: portal publico de carreiras.
- ``/careers/[slug]/jobs/[jobId]``: vaga publica.
- ``/api/demo/executive-pdf``: PDF comercial/executivo demo.

Autenticacao:

- ``/login``: login via credenciais.
- ``/invite/[token]``: aceite de convite.
- Middleware protege rotas internas.

Area interna:

- ``/dashboard``: cockpit inicial. Atualmente usa caminho rapido para evitar travamento por consulta pesada.
- ``/employees``: colaboradores.
- ``/employees/[employeeId]``: detalhe de colaborador.
- ``/requests``: solicitacoes internas de RH/service desk.
- ``/people/tasks``: tarefas operacionais.
- ``/people/onboarding``: onboarding.
- ``/people/offboarding``: offboarding.
- ``/people/compliance``: compliance.
- ``/people/calendar``: eventos e calendario de People Ops.
- ``/people/command-center``: central operacional.
- ``/people/agent-approvals``: aprovacoes de acoes de IA.
- ``/knowledge``: base de conhecimento.
- ``/chat``: Company Chat.
- ``/automations``: automacoes assistidas.
- ``/candidates``: candidatos.
- ``/candidates/[candidateId]``: detalhe de candidato.
- ``/applications/[applicationId]``: detalhe de aplicacao.
- ``/jobs``: vagas.
- ``/jobs/[jobId]``: detalhe de vaga.
- ``/pipeline``: pipeline.
- ``/interviews``: entrevistas.
- ``/interviews/[interviewId]``: detalhe de entrevista.
- ``/analytics``: analises.
- ``/communications``: comunicacoes.
- ``/ops/inbox``: inbox operacional.
- ``/ops/revenue``: revenue operations.
- ``/settings``: configuracoes.
- ``/settings/billing``: billing.

APIs relevantes:

- ``/api/auth/[...nextauth]``: Auth.js.
- ``/api/health``: health check simples.
- ``/api/ready``: readiness.
- ``/api/cron/jobs``: processamento por cron.
- ``/api/cron/billing``: rotinas de billing.
- ``/api/stripe/webhook``: webhook Stripe.
- ``/api/v1/desktop/*``: endpoints para cliente desktop.
- ``/api/interviews/[interviewId]/ics``: exportacao ICS de entrevista.

8. Dominios principais

Identity, organizacao e acesso**Arquivos importantes:**

- ``auth.ts``
- ``auth.config.ts``
- ``middleware.ts``
- ``lib/auth/current-user.ts``
- ``lib/auth/permissions.ts``
- ``lib/auth/permission-matrix.ts``
- ``lib/auth/organization-context.ts``

Responsabilidades:

- Login por credenciais.
- Sessao JWT.
- Usuario com organizacao principal.
- Memberships multi-org.
- Permissoes por papel.
- Protecao de rotas internas.
- Redirecionamento para login quando nao autenticado.

Modelos relacionados:

- ``Organization``
- ``User``
- ``OrganizationMembership``
- ``OrganizationInvite``
- ``Account``
- ``Session``
- ``VerificationToken``

Regra importante para qualquer IA que altere codigo:

Toda query de dado sensivel deve respeitar ``organizationId``. Nunca buscar registros internos apenas por ``id`` sem filtrar tenant.

Employees

Arquivos:

- ``modules/employees/queries.ts``
- ``modules/employees/service.ts``
- ``modules/employees/validators.ts``
- ``app/(app)/employees/page.tsx``
- ``app/(app)/employees/[employeeId]/page.tsx``

Funcionalidades:

- Diretorio de colaboradores.
- Perfil de colaborador.
- Status de ciclo de vida.
- Departamento, cargo, gerente e emails.
- Historico e relacoes com workflows, tarefas, requests, compliance e eventos.
- Assistente contextual e proximo passo recomendado em detalhes importantes.

Modelos:

- ``Employee``
- ``EmployeeCheckIn``
- ``PeopleEvent``
- Relacoes com ``PeopleTask``, ``HrRequest``, ``ComplianceRequirement``, workflows.

People workflows

Arquivos:

- ``modules/people-ops/queries.ts``
- ``modules/people-ops/service.ts``
- ``modules/people-ops/validators.ts``
- ``app/(app)/people/onboarding/page.tsx``
- ``app/(app)/people/offboarding/page.tsx``

Funcionalidades:

- Templates de onboarding/offboarding.
- Runs de workflow por colaborador.
- Steps com status.
- Criacao automatica de tarefas e eventos.
- Visao operacional de progresso.

Modelos:

- ``PeopleWorkflowTemplate``
- ``PeopleWorkflowTemplateStep``
- ``PeopleWorkflowRun``
- ``PeopleWorkflowStep``

People tasks

Arquivos:

- ``modules/people-tasks/queries.ts``
- ``modules/people-tasks/service.ts``
- ``modules/people-tasks/validators.ts``
- ``app/(app)/people/tasks/page.tsx``
- ``components/operations/people-tasks-workspace.tsx``

Funcionalidades:

- Lista de tarefas de People Ops.
- Status, prioridade, responsavel, prazo e comentarios.
- Triage inteligente: urgencia, risco, area responsavel, proxima acao e se pode resolver automaticamente.
- Bulk actions e interacoes mais rapidas nos workspaces.
- Acoes com IA como resolver, adiar, concluir, delegar ou comentar.

Modelos:

- ``PeopleTask``
- ``PeopleTaskComment``

HR requests / service desk

Arquivos:

- ``modules/hr-requests/queries.ts``
- ``modules/hr-requests/service.ts``
- ``modules/hr-requests/validators.ts``
- ``app/(app)/requests/page.tsx``
- ``components/operations/requests-workspace.tsx``

Funcionalidades:

- Fila de solicitacoes internas.
- Categoria, prioridade, status, solicitante, responsavel e SLA.
- Comentarios.
- Triage inteligente inline na lista.
- Sugestao de proximo passo.
- Resolver com IA.
- Knowledge relacionado.
- Preparacao de resposta e escalonamento.

Modelos:

- ``HrRequest``
- ``HrRequestComment``

Compliance

Arquivos:

- `modules/compliance/queries.ts`
- `modules/compliance/service.ts`
- `app/(app)/people/compliance/page.tsx`

Funcionalidades:

- Requisitos de compliance por colaborador.
- Status pendente, concluido, vencido etc.
- Policy rollout e aceite de politicas.
- Base para auditoria e aprovacoes.

Modelos:

- `ComplianceRequirement`
- `PolicyAcknowledgement`
- `PolicyRollout`

Knowledge**Arquivos:**

- `modules/knowledge/queries.ts`
- `modules/knowledge/ingestion-service.ts`
- `modules/knowledge/chunking.ts`
- `app/(app)/knowledge/page.tsx`

Funcionalidades:

- Base de conhecimento interna.
- Documentos, politicas, chunks e preparo para recuperacao por IA.
- Evidencias para respostas do chat.
- Uso contextual em solicitacoes, aprovacoes e chat.

Modelos:

- `KnowledgeDocument`
- `KnowledgeChunk`

Company Chat**Arquivos:**

- `app/(app)/chat/page.tsx`
- `app/(app)/chat/actions.ts`
- `components/chat/\*`
- `modules/company-chat/runtime.ts`
- `modules/company-chat/service.ts`
- `modules/company-chat/queries.ts`
- `modules/company-chat/tools.ts`

- `lib/validations/company-chat.ts`

Funcionalidades:

- Chat corporativo estilo ChatGPT.
- Composer fixo.
- Thread limpa.
- Sugestões acionáveis.
- Contexto recolhível.
- Respostas com propostas de ação.
- Cada resposta pode virar ação: criar tarefa, responder solicitação, mover etapa, abrir onboarding, gerar resumo, criar nota etc.
- Busca contexto em candidatos, vagas, aplicações, colaboradores, solicitações, tarefas, conhecimento, dashboard, analytics e compliance.

Papel correto do chat:

Ele não deve ser apenas uma tela separada. Ele deve operar o produto. Quando uma resposta sugerir algo executável, a UI deve apresentar um card de proposta com botão para aplicar, aprovar ou revisar.

Modelos:

- `ChatThread`
- `ChatMessage`
- Relação opcional com `AgentRun`.

AI Agent Runtime**Arquivos:**

- `modules/ai-agent/registry.ts`
- `modules/ai-agent/service.ts`
- `modules/ai-agent/queries.ts`
- `components/ai-agent/agent-approval-review-form.tsx`
- `app/(app)/people/agent-approvals/page.tsx`
- `types/company-chat.ts`

Funcionalidades:

- Registro central de ações que a IA pode executar.
- Preview antes da execução.
- Permissão por ação.
- Nível de risco.
- Aprovação obrigatória para ações sensíveis.
- AgentRun registra objetivo, payload, risco, status e resumo.
- AgentStep registra etapas internas.
- AgentApprovalRequest registra pedido de aprovação.
- AgentActionExecution registra execução e resultado.

Fluxo típico:

1. Chat ou painel contextual gera proposta.
2. Usuario clica para aplicar.
3. `applyAgentAction` busca definicao no registry.
4. Sistema valida permissao.
5. Sistema gera preview.
6. Se a acao exige aprovacao, cria AgentRun em `WAITING\_APPROVAL` e ApprovalRequest.
7. Se nao exige aprovacao, executa direto com registro de AgentActionExecution.
8. Aprovador pode aprovar/rejeitar em `/people/agent-approvals`.
9. Execucao altera dados reais usando services de dominio.

Acoes tipicas:

- Criar nota.
- Mover etapa.
- Criar tarefa.
- Atualizar status de tarefa.
- Criar solicitacao.
- Comentar ou responder solicitacao.
- Atualizar status de solicitacao.
- Iniciar onboarding/offboarding.

Regra de seguranca:

A IA nunca deve bypassar tenant, permissao, aprovacao, auditoria ou validacao de dominio.

AI helper layer**Arquivos:**

- `lib/ai/config.ts`
- `lib/ai/openai.ts`
- `lib/ai/resolve-assist.ts`
- `lib/ai/next-step.ts`
- `lib/ai/triage.ts`
- `lib/ai/stage-copilot.ts`
- `lib/ai/resume-parser.ts`

Funcionalidades:

- Configuracao de provider.
- Cliente OpenAI-compatible.
- Resolver com IA.
- Proximo passo recomendado.
- Triage inteligente de requests e tasks.
- Copilot de etapa.
- Parser de curriculos.

Triage:

`lib/ai/triage.ts` classifica urgencia, risco, area, proxima acao, possibilidade de auto-resolucao, razao e sugestao de automacao. A logica atual e heuristica/estruturada, nao necessariamente chamada LLM.

Automations e Watchtower

Arquivos:

- `modules/automations/queries.ts`
- `components/automations/\*`
- `app/(app)/automations/page.tsx`
- `lib/automation/job-rules.ts`
- `modules/watchtower/queries.ts`
- `modules/watchtower/service.ts`
- `modules/background-jobs/service.ts`
- `modules/background-jobs/processors.ts`
- `scripts/process-background-jobs.ts`

Funcionalidades:

- Visualizacao de automacoes.
- Regras sugeridas por contexto.
- Watchtower para sinais operacionais.
- Jobs de background para ingestao, scoring, alertas, lembretes e resumos.
- Processamento manual/cron por script e endpoint.

Modelos:

- `BackgroundJob`
- Enums de trigger e status.

Recruiting

Arquivos:

- `modules/recruiting/\*`
- `modules/recruiting-ops/queries.ts`
- `lib/jobs/queries.ts`
- `lib/candidates/queries.ts`
- `lib/applications/queries.ts`
- `lib/pipeline/queries.ts`
- `lib/interviews/queries.ts`
- `components/jobs/\*`
- `components/candidates/\*`
- `components/interviews/\*`
- `app/(app)/jobs/\*`
- `app/(app)/candidates/\*`
- `app/(app)/applications/\*`
- `app/(app)/pipeline/page.tsx`
- `app/(app)/interviews/\*`

Funcionalidades:

- Vagas.
- Candidatos.
- Aplicacoes.
- Pipeline.
- Etapas.
- Entrevistas.
- Feedback.
- Scorecards.
- Notas de hiring.
- Automacao de email e comunicacao.
- Portal publico de carreira.
- Parsing e scoring de curriculos.

Modelos:

- `Job`
- `JobCriterion`
- `Candidate`
- `Resume`
- `Application`
- `HiringNote`
- `Interview`
- `InterviewFeedback`
- `PipelineStage`
- `ApplicationStageHistory`
- `EmailTemplate`

Billing e Revenue Ops**Arquivos:**

- `lib/billing/\*`
- `app/(app)/settings/billing/page.tsx`
- `app/(app)/ops/revenue/page.tsx`
- `app/api/stripe/webhook/route.ts`

Funcionalidades:

- Planos.
- Status de billing.
- Upgrade requests.
- Uso.
- Stripe.
- Revenue operations.

Modelos:

- `BillingUpgradeRequest`

- Campos de billing em `Organization`.

Desktop

Arquivos:

- `apps/desktop/`
- `modules/desktop/queries.ts`
- `lib/auth/desktop-session.ts`
- `app/api/v1/desktop/\*`

Funcionalidades:

- Cliente nativo Tauri.
- Home executiva.
- Inbox.
- Tarefas.
- Requests.
- Calendario.
- Quick actions.
- Company chat.

9. Banco de dados e multi-tenancy

O banco usa Prisma e PostgreSQL.

Padrao principal de tenancy:

- A maioria dos registros operacionais possui `organizationId`.
- Queries devem filtrar por `organizationId`.
- Usuario pode ter memberships em mais de uma organizacao.
- `getActiveOrganizationCookie` define contexto ativo quando aplicavel.

Modelos mais sensiveis:

- `User`
- `Employee`
- `HrRequest`
- `PeopleTask`
- `KnowledgeDocument`
- `KnowledgeChunk`
- `ChatThread`
- `ChatMessage`
- `AgentRun`
- `AgentApprovalRequest`
- `AgentActionExecution`
- `AuditEvent`
- `BillingUpgradeRequest`

Diretriz para novas features:

- Sempre incluir tenant scope.
- Sempre validar permissao antes de mutar.
- Preferir services de dominio em vez de Prisma direto na UI.
- Para acoes de IA, passar pelo registry e agent runtime.

10. Interface e sistema visual atual

O produto passou por varias direcoes visuais, mas a direcao atual desejada e:

- Simples.
- Intuitiva.
- Rapida.
- Funcional.
- Pouco decorativa.
- Mais proxima de Vercel, ChatGPT, Notion, Linear e Trello.
- Menos experimental.
- Menos canvas/grafico.
- Menos bordas arredondadas exageradas.
- Menos frames e caixas desnecessarias.
- Mais clareza de navegacao, lista, detalhe e acao.

Componentes importantes:

- ``components/layout/harpia-system-shell-client.tsx``: shell principal e sidebar.
- ``components/navigation/*``: navegacao.
- ``components/operations/*``: workspaces de requests/tasks.
- ``components/chat/*``: experiencia do Company Chat.
- ``components/ai/*``: Resolve com IA, triage pill, contextual assistant, assisted create e next step.
- ``components/ui/*``: primitivos de UI.

Pontos de UX que importam:

- Sidebar legivel e recolhivel.
- Labels escondidas quando recolhida, mas previsiveis com tooltip.
- Filtros e views autoexplicativos.
- Empty states que orientam proximo passo.
- Feedback claro de loading, salvando, sucesso e erro.
- Tema claro e escuro consistentes.
- Chat estilo ChatGPT, com conversa fluindo sem obrigar o usuario a rolar ate o topo.

11. IA no produto

Harpia deve parecer inteligente em varios pontos, nao so no chat.

Camadas de IA:

- Triage inline em requests e tasks.
- Proximo passo recomendado em telas de detalhe.
- Resolver com IA em itens operacionais.
- Assistente contextual lateral.
- Criacao assistida de request, task, note, candidate e workflow.
- Chat corporativo que gera propostas de acao.
- Aprovacao premium para acoes sensiveis.
- Automacao assistida.
- Knowledge contextual mostrando fontes ou politicas relevantes.

Formato ideal de uma sugestao de IA:

- O que foi observado.
- Proximo passo recomendado.
- Por que isso importa.
- Acao rapida.
- Risco.
- O que vai mudar.
- Se precisa aprovacao.
- Como auditar ou desfazer.

Exemplos:

- Em uma solicitacao: responder, reatribuir, fechar ou escalar.
- Em uma tarefa: concluir, adiar ou delegar.
- Em um colaborador: registrar follow-up, iniciar onboarding ou abrir request.
- Em candidato: mover etapa, criar nota, agendar entrevista ou resumir perfil.

12. Seguranca, auditoria e confianca

O produto precisa parecer enterprise-ready.

Regras importantes:

- Toda acao sensivel deve ter permissao.
- Toda acao de IA deve ser rastreavel.
- Acoes de alto risco exigem aprovacao.
- Aprovacao deve mostrar acao proposta, motivo, registros afetados, antes/depois, nivel de risco e botoes aprovar/rejeitar.
- Nao armazenar segredo em documentacao.
- Nao expor dados de outra organizacao.
- Nao permitir que chat execute mutacao direta sem agent runtime.

Infra existente:

- `AuditEvent`
- `AgentRun`
- `AgentStep`
- `AgentApprovalRequest`

- ``AgentActionExecution``
- Permissoes por role.
- Health e readiness endpoints.
- Background jobs com status.

13. Como uma IA deve trabalhar neste codigo

Antes de alterar:

- Ler rota em ``app/``.
- Identificar modulo de dominio em ``modules/``.
- Verificar permissao exigida.
- Verificar schema Prisma.
- Procurar componentes ja existentes.
- Preservar tenant scope.

Ao implementar:

- Paginas devem ser finas.
- Regras de negocio devem ficar em ``modules/*/service.ts``.
- Leituras em ``queries.ts``.
- Validacao em ``validators.ts`` ou ``lib/validations``.
- UI em ``components/*``.
- Mutacoes de IA devem passar por ``modules/ai-agent/registry.ts`` e ``service.ts``.
- Background jobs devem passar pelo modulo de jobs.
- Evitar duplicar padroes de UI.

Ao validar:

- Rodar ``npm run typecheck``.
- Rodar ``npm run lint``.
- Rodar build quando viavel.
- Se build local cair com ``spawn EPERM``, verificar se typecheck/lint passaram e validar build cloud quando for deploy.

Coisas que uma IA nao deve fazer:

- Nao remover mudancas existentes sem confirmar.
- Nao quebrar rotas protegidas.
- Nao criar features fake que so parecem funcionar.
- Nao colocar dados reais ou segredos em docs.
- Nao ignorar permissions.
- Nao consultar Prisma sem ``organizationId`` em dados multi-tenant.
- Nao fazer IA alterar dados sensiveis sem aprovacao.

14. Estado recente importante

O dashboard teve problema de carregamento em producao possivelmente relacionado a consultas pesadas e pool de banco. Para reduzir risco, `/dashboard` foi ajustado para renderizar um estado rapido sem depender de muitas queries no primeiro carregamento. Isso deve ser tratado como decisao de estabilidade temporaria: o ideal futuro e carregar dados vivos por endpoint/cache dedicado ou read model leve.

O produto tem muitas mudancas recentes focadas em:

- Company Chat mais parecido com ChatGPT.
- Acoes de IA a partir das respostas.
- Triage inteligente.
- Assistente contextual.
- Criacao assistida.
- Automacoes.
- Aprovacoes premium.
- Workspaces mais intuitivos.
- Limpeza visual.

Se outra IA continuar o trabalho, ela deve primeiro conferir `git status`, porque pode haver varias mudancas nao commitadas dependendo do momento.

15. Arquivos mais importantes para entender rapido

Produto e marca:

- `README.md`
- `lib/brand.ts`
- `docs/brand-harpia.md`

Auth e permissao:

- `auth.ts`
- `auth.config.ts`
- `middleware.ts`
- `lib/auth/current-user.ts`
- `lib/auth/permission-matrix.ts`
- `lib/auth/permissions.ts`

Shell e UX:

- `components/layout/harpia-system-shell-client.tsx`
- `components/operations/requests-workspace.tsx`
- `components/operations/people-tasks-workspace.tsx`
- `components/chat/company-chat-scroll-area.tsx`
- `components/chat/company-chat-composer.tsx`

IA:

- `lib/ai/openai.ts`
- `lib/ai/triage.ts`
- `lib/ai/next-step.ts`

- ``lib/ai/resolve-assist.ts``
- ``modules/company-chat/runtime.ts``
- ``modules/company-chat/tools.ts``
- ``modules/ai-agent/registry.ts``
- ``modules/ai-agent/service.ts``

Dominios:

- ``modules/employees/*``
- ``modules/hr-requests/*``
- ``modules/people-tasks/*``
- ``modules/people-ops/*``
- ``modules/knowledge/*``
- ``modules/compliance/*``
- ``modules/recruiting/*``
- ``modules/background-jobs/*``
- ``modules/automations/*``

Banco:

- ``prisma/schema.prisma``
- ``prisma/seed.ts``

16. Prompts uteis para outra IA

Prompt para entender o projeto:

Leia ``docs/harpia-ai-context.md``, ``README.md``, ``prisma/schema.prisma``, ``auth.config.ts``, ``modules/ai-agent/registry.ts``, ``modules/company-chat/runtime.ts`` e os arquivos da rota que eu pedir. Antes de alterar código, explique o domínio envolvido, os dados tocados, as permissões necessárias e como vai validar.

Prompt para implementar feature:

Implemente a feature preservando a arquitetura atual: página fina em ``app/``, regra de negócio em ``modules/<dominio>/service.ts``, leitura em ``queries.ts``, validação com Zod quando houver input, UI em ``components/``, tenant scope por ``organizationId``, permissão antes de mutar e validação final com ``typecheck`` e ``lint``.

Prompt para IA/acesso seguro:

Qualquer ação gerada por IA que altere dados reais deve passar pelo agent runtime. Use ``modules/ai-agent/registry.ts`` para registrar tipo, permissão, risco, preview e executor. Se for alto risco, exija aprovação e registre `AgentRun`, `AgentStep`, `AgentApprovalRequest` ou `AgentActionExecution`.

17. Resumo final para onboarding de IA

Harpia é um SaaS de People & Internal Operations OS construído com Next.js, React, TypeScript, Prisma e PostgreSQL. Ele combina RH operacional, service desk interno, tarefas, workflows, knowledge base, chat corporativo, automações, aprovações e recruiting em uma experiência única. A diferença do produto é integrar IA ao fluxo operacional: a IA recomenda próximos passos, classifica filas, prepara respostas, cria registros, resume contexto e propõe ações reais, mas com permissão, tenant isolation, auditoria e aprovação quando necessário.

Para continuar o projeto com qualidade, uma IA deve entender que Harpia não é apenas um dashboard bonito nem apenas um chatbot. É um sistema de operação interna onde cada módulo precisa ser rápido, claro, confiável e acionável. A prioridade de produto é tornar o trabalho diário de People Ops mais simples: ver o que importa, decidir rápido, executar com segurança e manter rastreabilidade.